

OnSale — справочник вызовов

Два вида интерфейсов: **server actions** (вся запись данных) и несколько **HTTP-маршрутов** `/api/*`. Чтение данных идёт напрямую из серверных компонентов (`page.tsx`) через Prisma и `src/lib/stats.ts` и отдельного API не имеет.

1. Соглашения server actions

- Файлы: `src/app/actions.ts`, `src/app/price-actions.ts`; вызываются из клиентских компонентов как обычные асинх-функции.
- Возврат: `ActionResult<T> = { ok: true, ...T } | { ok: false, error: string }`. Текст `error` можно показывать пользователю как есть.
- Порядок внутри каждого действия: сессия → роль → zod-валидация → проверка прав на запись → транзакция (изменение + журнал) → сброс кэша страниц.
- Клиент: `const r = unwrap(await changeOrderStatus(id, "SHIPPED"))` — при `ok:false` бросит `Error(error)`, который показывается через `toast.error`.
- Общие ошибки для всех действий: «Сессия истекла — войдите заново», «Недостаточно прав», «Запись не найдена — обновите страницу», «Данные одновременно меняет другой пользователь. Повторите операцию.», «Внутренняя ошибка...».
- Все изменения заказов/оплат сериализуются через `writeTx` (очередь записи, см. ARCHITECTURE §7).

Обозначения ролей: B — BUYER, A — AGENT, S — SELLER, C — COURIER, ADM — ADMIN.

2. Заказы

Действие	Роли	Аргументы	Возвращает
<code>createOrder(buyerId, items, note?)</code>	B, A, ADM	<code>items: {productId, qty}[]</code>	<code>{ number }</code>
<code>submitOrder(orderId)</code>	B, A, ADM		<code>{}</code>
<code>cancelOrder(orderId)</code>	B, A, ADM		<code>{}</code>
<code>changeOrderStatus(orderId, status)</code>	S, ADM, C		<code>{}</code>
<code>bulkChangeStatus(orderIds, status)</code>	S, ADM	до 5000 id	<code>{ changed, skipped: string[] }</code>
<code>editOrderItems({orderId, items, reason})</code>	S, ADM	<code>items: {orderId, qty}[]</code>	<code>{ total, removed, changes }</code>
<code>markOrderDeleted(orderId, deleted)</code>	ADM		<code>{}</code>

createOrder. Покупатель заказывает только за себя; агент — только за закреплённых покупателей (`agentId` заказа = агент); админ — за любого. Покупатель не должен быть заблокирован. Повторы позиций суммируются, позиции с `qty=0` отбрасываются, нужна хотя бы одна. Товар должен существовать и не быть удалённым (иначе «Некоторые товары сняты с продажи...»). Цена и название копируются в позицию (снимок). Создаётся заказ в статусе `NEW` с записью в `OrderStatusLog`. Номер — `max+1`; при гонке — до 5 повторов.

submitOrder / cancelOrder. Доступ только к своему заказу (покупатель) или заказу своего покупателя (агент). Передача в работу — только из `NEW` в `ENTERED`. Отмена — только из `NEW / ENTERED`.

changeOrderStatus. Допустимость перехода определяет `transitionError` (ARCHITECTURE §8), в том числе: курьер — только `SHIPPED→DELIVERED`; в `PAID` и из `PAID` вручную нельзя; `CANCELLED` — без оплат, восстановление только ADM. Каждая смена пишется в `OrderStatusLog`.

bulkChangeStatus. Применяет `changeOrderStatus` к каждому заказу в **отдельной** транзакции; заказы, которым переход запрещён, пропускаются и возвращаются в `skipped` в виде "`№123: причина`", остальные меняются.

editOrderItems. Только `NEW/ENTERED/ASSEMBLED`. `reason` обязателен (из справочника `OrderEditReason` или свой текст). Пересчитывает `total`, пишет `order_edited` в `OrderAuditLog`, при необходимости пересчитывает статус `PAID`. Ошибки: «Нет изменений», «Нельзя удалить все позиции — отмените заказ», «По заказу уже оплачено ... больше новой суммы».

3. Оплаты (S, ADM)

Действие	Аргументы	Возвращает
<code>createPayment({buyerId?, orderId?, amount, method?, note?})</code>	<code>method</code> : <code>card</code> (по умолч.), <code>cash</code> , <code>invoice</code>	<code>{}</code>
<code>correctPayment({paymentId, amount?, method?, note?, reason?})</code>	<code>reason</code> ≥ 3 символов	<code>{}</code>
<code>deletePayment(paymentId, reason)</code>	<code>reason</code> ≥ 3 символов	<code>{}</code>

- `createPayment` **c** `orderId`: заказ в статусе `ASSEMBLED/SHIPPED/DELIVERED`, сумма ≤ остатка долга по заказу (иначе ошибка с точным остатком). Если `buyerId` передан, он должен совпадать с покупателем заказа. После записи — `syncPaymentStatus` (при полной оплате заказ становится `PAID`, в `OrderStatusLog` запись «Полная оплата»). **Без** `orderId`: нужен `buyerId` покупателя; запись в `AuditLog`.
- `correctPayment` / `deletePayment`: продавец — только оплаты, внесённые сегодня (по `Asia/Novosibirsk`); админ — любые. Пишут `payment_edited` / `payment_deleted` в `OrderAuditLog` (если оплата привязана к заказу) или в `AuditLog`. После изменения статус заказа пересчитывается: оплат стало меньше суммы → откат из `PAID`; оплат стало больше суммы → отказ.
- Все суммы: > 0, ≤ 1 000 000 000, округление до копеек.

4. Пользователи (только ADM)

Действие	Аргументы / правила
<code>createUser({email, name?, password, role, agentId?, address?, phone?, comment?, deferral?})</code>	<code>email</code> уникален (без учёта регистра), пароль 8–128 символов, <code>agentId</code> — только для BUYER и только пользователь-AGENT, <code>deferral</code> 0–365 дней
<code>updateUser(id, {name?, role?, agentId?, password?, address?, phone?, comment?, deferral?})</code>	смена роли: не себе, не последнему активному админу; смена пароля увеличивает <code>sessionVersion</code> (старые сессии выбрасываются); пишет <code>AuditLog</code> : <code>user_updated</code>
<code>setUserBlocked(id, blocked)</code>	блокировка: не себя, не последнего админа; увеличивает <code>sessionVersion</code> ; <code>AuditLog</code> : <code>user_blocked/unblocked</code>
<code>deleteUser(id)</code>	нельзя: себя, последнего админа, пользователя с заказами/оплатами/прайс-листами (его блокируют); <code>AuditLog</code> : <code>user_deleted</code>

Все четыре возвращают `{}`; в журнал `createUser` пишет `user_created`.

5. Каталог (только ADM)

Действие	Аргументы	Возвращает
<code>createProduct({name, article?, unit?, price?, stock?, manufacturer?, categoryId?})</code>	пустой артикул → «АПТ-N+1»; занятый → ошибка	<code>{ id, article }</code>
<code>updateProduct(id, {name?, price?, stock?, unit?, categoryId?, manufacturer?})</code>		<code>{}</code>
<code>deleteProducts(ids)</code>	мягкое удаление (скрыть)	<code>{ count }</code>
<code>restoreProducts(ids)</code>	вернуть	<code>{ count }</code>
<code>hardDeleteProducts(ids)</code>	физическое удаление; товары с историей заказов пропускаются	<code>{ count, skipped, skippedMessage? }</code>
<code>createCategory(name) / renameCategory(id, name) / deleteCategory(id)</code>	удаление отвязывает товары (категория → null)	<code>{ id } / {}</code>
<code>mergeCategories(sourceId, targetId)</code>	товары переносятся в target, source удаляется	<code>{}</code>
<code>createEditReason(name) / deleteEditReason(id)</code>	справочник причин корректировки	<code>{}</code>

Изменение цены не затрагивает уже созданные заказы (снимок в `OrderItem`).

6. Импорт прайс-листа (только ADM; `price-actions.ts`)

Эти два действия возвращают `{ ok:true, ... } | { ok:false, error }` (без `UserError`).

Действие	Что делает	Возвращает
<code>stagePricelist(formData)</code>	поле <code>file</code> (CSV/XLSX ≤ 15 МБ) → разбор без записи в БД	<code>{ ok, data: { encoding, delimiter?, totalRows, skipped, products[], categories[], existingCategories[] } }</code>
<code>applyPricelist({products, categoryChoices})</code>	повторная валидация каждой строки; одна транзакция (таймаут 120 с); <code>`categoryChoices[имя]` = "create" \</code>	имя_существующей \ "" <code>{ ok, created, updated, categoriesCreated, articlesGenerated }</code>

Поля строки: `article` (может быть пустым), `name` (обязательно), `unit`, `price` (≥ 0), `stock` (целое ≥ 0), `category`, `manufacturer`. Максимум 20 000 строк.

7. Профиль

`updateBuyerProfile({address?, phone?, comment?})` — роль BUYER, меняет только свой профиль → `{}`.

8. HTTP-маршруты

Маршрут	Метод	Доступ	Назначение	
/api/health	GET	открыт	200 {ok:true, db:"ok", ts}; 503 {ok:false, db:"error"}, если не отвечает БД (SELECT 1). Используют Docker healthcheck и watchdog	
/api/auth/*	GET/POST	NextAuth	вход (/api/auth/callback/credentials), выход, сессия (/api/auth/session), CSRF	
/api/auth-status	POST {email}	открыт	{blocked, retryAfterSec, attemptsLeft} — состояние лимита попыток для формы входа; без секретов	
/api/price-template	GET `?format=csv\`	xlsx`	ADMIN (иначе 403)	шаблон прайс-листа (CSV — UTF-8 с BOM, разделитель ;)

9. Как добавить новое действие

1. Добавьте zod-схему в `src/lib/validation.ts`.
2. В `actions.ts` напишите функцию по каркасу `run(async () => { assertRole → parse → writeTx → журнал → revalidate })`. Бизнес-правила, не требующие БД, выносите в чистые функции (как `order-rules.ts`) и покрывайте тестами.
3. На клиенте вызывайте через `unwrap()` и показывайте ошибки через `toast`.
4. Добавьте тест в `src/actions.integration.test.ts` (проверьте: чужая роль, чужая запись, неверные данные, гонка).
5. Если действие меняет данные, видимые на страницах, добавьте `revalidatePath`.